SZÉCHENYI EGYETEM
UNIVERSITY OF GYŐR

Sim2Real concept and typical gaps between simulation and reality

Audi Development Camp 2026

Rudolf Krecht

JÁRMŰIPARI KUTATÓKÖZPONT
VEHICLE INDUSTRY RESEARCH CENTER

SZÉCHENYI EGYETEM
UNIVERSITY OF GYŐR

What is Sim2Real?

Part 1

JÁRMŰIPARI KUTATÓKÖZPONT
VEHICLE INDUSTRY RESEARCH CENTER

JÁRMŰIPARI KUTATÓKÖZPONT
VEHICLE INDUSTRY RESEARCH CENTER

SZÉCHENYI EGYETEM
UNIVERSITY OF GYŐR

Agenda

1	**What is Sim2Real**	Why it matters
2	**What is the reality gap?**	Dynamics, contact, sensors, latency
3	**Failure modes**	When a policy overfits or exploits the simulator
4	**Bridging strategies**	Better models, domain randomization, adaptation

Where Sim2Real sits in the pipeline

STAGE 1

Model the world

URDF / MJCF, meshes, actuators, sensors

STAGE 2

Train in sim

PPO / SAC over millions of steps, GPU-parallel

STAGE 3 — TODAY

Cross the gap

Get the policy from sim onto the real robot

STAGE 4

Deploy on hardware

Real robot, real world, real consequences

JÁRMŰIPARI KUTATÓKÖZPONT
VEHICLE INDUSTRY RESEARCH CENTER

SZÉCHENYI EGYETEM
UNIVERSITY OF GYŐR

What is Sim2Real?

Sim2Real (simulation-to-real) is the discipline of taking a policy, model, or controller that was **trained inside a simulator** and making it work on the **physical robot**, even though the two worlds are never identical.

TRAIN

In the simulator

Cheap, fast, safe, parallel. Millions of episodes on a GPU in an afternoon.

TRANSFER

Across the gap

Robustify the policy so it survives the mismatch between the model and the world.

DEPLOY

On real hardware

No re-training on the robot, no human in the loop — the policy just runs.

Why is it important?

SPEED

Millions of steps in an hour

On Isaac Gym, ANYmal has been trained to walk in about **4 minutes** of wall-clock time using a single GPU with 4096 parallel envs.

SAFETY

Fall a million times — for free

A humanoid that falls in sim costs nothing. On hardware, one bad fall can be six weeks of repair, thousands of euros, and a torn tendon.

SCALE

Terrains that don't exist yet

Curricula over thousands of procedurally generated terrains, wind loads, and payloads that you could never build in one lab.

RESET

Ground truth on tap

Perfect state, perfect labels, perfect resets, perfect reproducibility. None of these exist on the physical robot.

The aim: zero-shot transfer

The dream of Sim2Real is **zero-shot transfer** — you train once, in simulation, and the policy just *works* on the physical robot the first time you press "run".

- No hand-tuning
- No on-robot data collection
- No calibration episodes

TRANSFER LEVELS

0 **Zero-shot**
Deploy as-is. Nothing on the robot.

1 **Few-shot / online adapt**
A short adaptation phase on hardware (seconds–minutes).

2 **Fine-tuning**
Continue RL on the real robot — hours to days.

3 **Sim-in-the-loop retraining**
Real data flows back to update the simulator itself.

The reality gap

reality = simulation +reality gap

A policy trained in a simulator has been optimized **against a model**, not against the world. The moment it lands on hardware, everything the model failed to capture becomes a source of error — hidden friction, sensor drift, delayed torques, an unmodelled ground vibration.

This unmodelled residual is the **reality gap**, and every Sim2Real technique in the rest of this lecture exists to shrink it, hide it, or force the policy to survive it.

Rule of thumb — if a behaviour appears only in simulation and disappears on the robot, either the sim is wrong *or* the policy is exploiting a sim artifact. Both mean the same thing: **it must be fixed.**

The reality gap

SIMULATION MDP

M_{sim}

$< S, A, \mathbf{P_{sim}}, R, \gamma >$

$P_{sim}(s' \mid s, a)$ is what the simulator says will happen.

REAL-WORLD MDP

M_{real}

$< S, A, \mathbf{P_{real}}, R, \gamma >$

P_{real} is the true world — never observed in full.

THE SIM2REAL PROBLEM

Given only access to samples from **M_{sim}**, learn a policy π that maximises expected return on **M_{real}**:

$$\pi^* = arg \max_{\pi} \mathbb{E}_{\tau \sim \pi, P_{real}} \left[\sum_t \gamma^t R_t \right]$$

Working assumption: P_{sim} and P_{real} differ only in *bounded, structured* ways — parameters like friction, mass, latency, and sensor noise. If the differences are unbounded or unstructured (e.g. a mid-flight morphology change), Sim2Real by itself will not save you.

Notation: S = state space, A = action space, P = transition dynamics, R = reward, γ = discount, τ = trajectory.

Main difference sources

Property	Simulation	Real robot
State	Perfect ground truth for every joint	Noisy, partial, occasionally missing
Reset	Free, deterministic, instant	An operator lifts the robot
Time	Faster than real-time, GPU-parallel	Real-time, one robot at a time
Failure cost	Zero — the robot respawns	€, weeks of downtime, safety incident
Dynamics	Whatever the physics engine implements	The laws of physics
Actuator	Ideal torque source	Backlash, friction, current limits, thermal drift
Communication	Zero latency	Multi-ms buses, jitter, dropped packets
World	The one you built in MJCF	Wet floors, cables, curious humans

When is a Sim2Real transfer successful?

1 **Task success rate on hardware ≥ target**

Over N independent trials, the robot completes the task at least X% of the time — under conditions the policy has never seen.

2 **Transfer gap is small**

The drop in return between sim and real is bounded. A policy that gets 100% in sim and 5% on hardware has not transferred — it has merely deployed.

3 **Behaviour is robust, not lucky**

Small perturbations — a push, a new floor, a different battery voltage — do not collapse the policy. If one bad trial is enough to break everything, transfer has not happened yet.

Short history

Sim2Real is a decade-old term, but the problem is as old as robot simulation itself



Two big shifts: **2017** — the field stops trying to build a perfect simulator and starts training on many bad ones (domain randomization). **2021** — the field stops trying to guess the real world and starts letting the policy *identify* it online (adaptation).

SZÉCHENYI EGYETEM
UNIVERSITY OF GYŐR

The reality gap

Part 2

JÁRMŰIPARI KUTATÓKÖZPONT
VEHICLE INDUSTRY RESEARCH CENTER

A taxonomy of gaps

01 · DYNAMICS
Rigid-body idealisation
Real links flex, oscillate, hum.

02 · ACTUATORS
Ideal motors, real gearboxes
Backlash, hysteresis, current limits.

03 · CONTACT
Solver-dependent physics
Point vs. area, penetration, stiction.

04 · FRICTION
One μ ≠ every floor
Dust, moisture, wear.

05 · PROPRIOCEPTION
Noisy IMU & encoders
Bias, drift, quantization.

06 · VISION
Renderer ≠ CMOS
Lighting, exposure, motion blur.

07 · LATENCY
No such thing as t=0
Sensing, compute, comms delays.

08 · PERTURBATIONS
Wind, cables, humans
Never in the sim world.

09 · MORPHOLOGY
CAD ≠ built robot
Mass, inertia, link lengths off.

10 · ENVIRONMENT
Terrain in the wild
Slopes, deformables, stairs.

11 · OBSERVATION
Different feature spaces
Sim gives features real has to estimate.

12 · TEMPORAL
Discretisation artefacts
dt in sim vs. control rate on robot.

The rigid-body idealisation

Most simulators represent a robot as a chain of **perfectly rigid links** connected by ideal joints. This makes the equations of motion cheap and stable — a rigid-body dynamics solver only has to invert a mass matrix.

A real robot is not rigid. Aluminium arms bend. Rubber feet compress. Long serial chains oscillate like a fishing rod when the base accelerates. None of these degrees of freedom exist in the model — so the policy is trained in a world that quite literally has fewer dimensions than the one it will run in.

TELL-TALE ON HARDWARE

The robot "rings" when you command a step change

In sim the joint tracks the setpoint instantly. On hardware you hear a whine and the base wobbles for a few hundred milliseconds — a mode the policy was never told about.

Fix directions — add flexible-body modes to the URDF (expensive); or, more common, add **heavy action-smoothing** and **joint-velocity penalties** to the reward so the policy never commands step changes the real robot cannot follow.

Ideal vs. real motors

SIMULATOR

Ideal torque source

- You command torque T, the joint applies exactly T.
- Response time: one physics step.
- Efficiency: 100%.
- Symmetric behaviour in both directions.
- No temperature. Ever.

REAL ACTUATOR

A messy dynamical system

- Command T, get T minus friction, backlash, PWM ripple.
- Response: PID loop with its own delay and bandwidth.
- Torque saturates at current limit — non-linearly with speed.
- Gearbox hysteresis: dead-band around zero.
- Motor heats up, resistance rises, torque drops.

Why this is the #1 gap for legged robots — contact events send huge, brief torque commands through the joints. A sim that turns them into perfect impulses trains a policy that stomps; the same policy on hardware saturates, overshoots, and falls.

Contacts become solver choices

Every physics engine handles contact differently. MuJoCo uses a **soft, convex** formulation that lets bodies interpenetrate slightly. Bullet, PhysX, ODE each pick different trade-offs between speed, stability and physical accuracy.

A quadruped foot on the floor is not a single point pressed into a plane. It is a patch with distributed pressure, edge effects, and micro-slips. Every simulator picks an approximation to this; none of them get all of it right.

WHERE IT BITES

- **Impulse-driven contact**
- **Interpenetration**

  Soft contact lets sim feet sink a millimetre — a physical foot cannot.
- **Stiction**
- **Edge collisions**

> Try the same policy in **MuJoCo** and **PhysX** — the disagreement between the two simulators is a lower bound on the disagreement either has with reality.

Friction coefficient

In sim you set friction=1.0 once and forget. On hardware, μ changes with the surface, the temperature, the humidity, the dust — and most importantly, with the direction of motion.

POLISHED LAB FLOOR

μ ≈ 0.3

Where most demos are filmed. Rubber feet slide.

CARPET / GRAVEL

μ ≈ 0.7 – 1.0

Grippy but with local heterogeneity — every step is different.

WET TILE / OIL PATCH

μ < 0.1

The number the demo-room policy has never seen.

Standard fix — **randomize μ** during training over [0.05, 1.5], and randomize the tangential/normal ratio too. A policy trained on μ = 1.0 alone will fail on the first wet lobby floor it meets.

The IMU that isn't telling the truth

Sensor	What sim gives you	What hardware actually gives you
Joint encoder	Exact θ, exact $\dot{\theta}$	Quantized θ, $\dot{\theta}$ from finite difference (noisy!), backlash-corrupted near zero
IMU accelerometer	Exact body acceleration	Zero-g bias that drifts with temperature, high-frequency chassis vibrations, ~0.05 m/s^2 noise
IMU gyroscope	Exact body ω	Bias walk (~°/s per hour), scale-factor error, cross-axis coupling
Orientation estimate	Exact roll/pitch/yaw	Filtered fusion — good pitch/roll, unbounded yaw drift, occasional glitches during hard impacts
Force/torque (foot)	6-DOF wrench, noise-free	Expensive, thermally sensitive, prone to bias jumps after a hard landing
Motor current	Exact applied torque	Only a proxy for torque — depends on temperature, back-EMF, drive electronics

Rule of thumb — if your sim observation is joint_pos = qpos, you are training on a fantasy. Wrap every observation with a noise + delay + bias model that matches the hardware datasheet.

The renderer is not a CMOS sensor

SYNTHETIC RGB / DEPTH

Uniform, sharp, well-lit

Perfect focus, no motion blur, flat lighting, textures the artist picked. Depth is exact metric distance — no missing pixels.

REAL CAMERA / DEPTH

Everything but uniform

Auto-exposure changes gain mid-episode. Rolling-shutter warp during fast motion. Depth returns NaN on shiny or dark surfaces.

- **Lighting** — sim uses point/directional lights; the world has skylight, shadows, flicker.
- **Motion blur** — hardware images blur under fast head motion; sim does not.
- **Depth holes** — glass, black rubber, direct sun all break stereo/ToF.
- **Texture domain** — an unseen wallpaper is out of distribution for a CNN.
- **Rolling shutter** — each row of the sensor sampled at a different time.

The vision gap is the reason domain randomization was invented — Tobin et al., 2017 trained an object detector on scenes with randomised textures, lights and camera poses, and it transferred zero-shot.

Nothing on a real robot happens at t = 0

CONTROL LOOP — ROUND-TRIP



In sim, the policy sees the current state and applies its action inside the same physics step. The loop is instantaneous. On hardware, the policy is looking at data that is already **10–20 ms stale** and its action lands another **5–10 ms** later.

For a walking humanoid at 50 Hz, that is more than a whole control step of dead time. Ignore it and the policy oscillates.

STANDARD FIX

Measure the true delay on hardware, then delay both observations **and** actions in sim by that amount plus 30–50% jitter. The policy learns to control a system it hasn't quite seen yet.

Wind and cables

Simulators are hermetic. Nothing enters the world you did not put there. The physical world is the opposite — full of small forces you never wrote down.

A tether cable pulls the base by half a Newton. A ventilation vent pushes air across the torso. Each of these is small in isolation and lethal to a policy that never saw external forces during training.

COMMON "PUSH" CURRICULA

- **Random torso wrench**
  Apply 5–150 N for 0.2 s at random intervals.
- **Payload changes**
  Add ±5 kg of ballast to random links each episode.
- **Terrain gusts**
  Time-varying horizontal force on the base.
- **Ground vibration**
  Small oscillation of the whole ground plane.

> A policy that has been kicked in sim will forgive the world for kicking it. A policy that has never been kicked in sim will fall the first time the tether tugs.

The CAD is a wish, the robot is a compromise

Your URDF was exported from a CAD file that described the robot as designed. The robot on the bench was **built** — with tolerances, extra cable bundles, thermal shims, and repairs after last month's crash.

Individual link masses can be off by 5–15%. The inertia tensor can be off by more. The centre of mass moves a few millimetres. For a static arm this hardly matters; for a humanoid balancing on one foot, it decides whether the policy stays upright.

TYPICAL MODEL ERRORS

Link mass — ± 5–15%

Inertia tensor — ± 20% or more

Centre of mass — ± 5–20 mm

Link length — ± 1–3 mm per link

Joint zero — ± 0.5° after calibration

Assembly play — a few mrad backlash

Standard fix — randomize link masses, inertias, and CoM offsets during training. Weigh the actual robot's links if you can — even a rough measurement beats trusting the CAD.

Terrain the sim has never rendered

FLAT GROUND

The demo floor

What most sim environments default to. Perfect plane, one μ, no obstacles. Easy to train, useless in the wild.

Failure mode: policy has no idea what a stair is.

STRUCTURED TERRAIN

Stairs, ramps, gaps

Curriculum of parametric terrains — step height, slope angle, gap width. The world of every recent quadruped paper.

Failure mode: real stairs have variable riser heights and a nose.

DEFORMABLE / GRANULAR

Sand, mud, grass

Where rigid-body sim breaks entirely. The foot sinks, the ground yields, μ becomes velocity-dependent.

Failure mode: no fix without a completely different physics engine.

Every deployment environment the policy will meet needs to appear in the training distribution — sim2real cannot make up terrain the policy has never seen.

Sim gives you features real has to estimate

A common Sim2Real bug is subtle: the policy's observation vector **looks the same on paper**, but the two halves are populated by different mechanisms. In sim, the base-linear-velocity comes straight from qvel. On the robot, no such sensor exists — it has to be estimated from an IMU and joint encoders, and that estimate can be biased, delayed, and completely wrong during flight phases.

Observation field	In sim, comes from	On robot, comes from
base_lin_vel	qvel[0:3] — perfect	EKF estimate — biased in flight phases
base_ang_vel	qvel[3:6] — perfect	Gyro with bias walk
projected_gravity	R × [0,0,-1] — exact	Filter fusion of accel+gyro — noisy under impact
joint_pos, joint_vel	qpos, qvel — perfect	Quantized encoder + finite-difference velocity
foot_contact	Contact array from solver	Torque threshold or F/T sensor with false positives

SZÉCHENYI EGYETEM
UNIVERSITY OF GYŐR

Failure modes

Part 3

JÁRMŰIPARI KUTATÓKÖZPONT
VEHICLE INDUSTRY RESEARCH CENTER

Overfitting to the simulator

The policy has learned the **one** world it was trained on — a single friction value, a single motor model, a single centre-of-mass — and it works perfectly there. Ship it to hardware where any of those quantities differ, and behaviour degrades gracelessly.

This is classic supervised-learning overfit, only worse — the training distribution had a single sample of every physical parameter.

SYMPTOMS

- Perfect in sim, brittle on hardware
- Works only in the demo room, never elsewhere
- Breaks when a joint is a bit sticky
- Breaks when the battery is at 20% instead of 80%
- Two robots of the same model behave differently

The cure — widen the training distribution. Randomize every physical parameter you can think of, at ranges that cover — and slightly exceed — the plausible real-world values. A policy that succeeds across a *family* of dynamics is much more likely to succeed on the one specific dynamics of your robot.

Exploiting the simulator

The subtler and more dangerous cousin of overfitting. The policy has found a strategy that only works because of a **bug or approximation in the physics engine** — a hack that pays reward in sim but is physically impossible in reality.

RL agents are excellent at this. They will happily wedge a foot through the ground, oscillate a joint at 500 Hz, or exploit a small solver interpenetration to "climb" one millimetre per step.

FAMOUS EXAMPLES

- A quadruped that shakes its head to add "free" energy
- A humanoid that vibrates its foot to gain thrust from soft contact
- An arm that uses object interpenetration to "grip"
- A biped that walks by bouncing on a solver artifact

Diagnostic — the policy's return graph looks great. Playback in the viewer looks … *slightly wrong*. Impossibly fast joint motion, weirdly stable single-foot stances, or high-frequency oscillations. Trust your eyes: if it looks like the robot is cheating, it probably is.

ANYmal



The first attempts to train ANYmal in generic rigid-body sim **failed on the physical robot**. Hwangbo et al. (2019) traced it to the joints — ANYmal uses **series-elastic actuators** whose torque response is a small nonlinear dynamical system, not an ideal source.

Their fix: train a supervised neural network — an **actuator net** — on real motor data (joint state → torque), then plug that network into the physics loop as the actuator model. The RL policy sees the same nonlinear behaviour it will meet on hardware.

RESULT

The first RL policy to walk a real quadruped in the wild

Published in *Science Robotics*, 2019. Established the pattern: **learn the sim from real data** for the components you can't model analytically.

Lesson — when a component is too complex to model analytically, don't try harder — **identify it from data**. The actuator net is one instance of a much broader pattern: *learned components inside a physics simulator*.

Source: Hwangbo et al., "Learning agile and dynamic motor skills for legged robots", Science Robotics 4(26), 2019.

JÁRMŰIPARI KUTATÓKÖZPONT
VEHICLE INDUSTRY RESEARCH CENTER

SZÉCHENYI EGYETEM
UNIVERSITY OF GYŐR

Cassie



Early Cassie sim-trained policies would walk perfectly in MuJoCo, then arrive on the robot and produce a **high-frequency knee chatter** that made the robot hum audibly at ~30 Hz. Nothing in the sim predicted it.

Cause: the policy had learned to command joint-velocity setpoints that changed sign every control step — perfectly stable in an ideal actuator, but excited a mechanical resonance in the real leg. The audible whine is the reality gap made audible.

THE FIX

Penalize action rate in the reward

Adding a term proportional to $\|a_t - a_{t-1}\|^2$ to the reward is now a **near-universal ingredient** of legged-robot Sim2Real recipes.

Lesson — any action pattern the sim is willing to produce but the hardware cannot execute cleanly is a bug waiting to happen. Reward smoothness in sim, and the policy will act smoothly on the robot.

In-hand manipulation: the friction cheat

Early sim-trained hand policies rotating a cube would apparently succeed — the cube followed the target orientation. Deployed on the real Shadow hand, the same policy dropped the cube within seconds.

Diagnosis: the sim's soft-contact solver quietly allowed the finger to **penetrate the cube** by a fraction of a millimetre and use the resulting virtual force as extra grip. The policy learned to depend on it. On hardware, the fingers meet a hard, non-penetrable surface — the "grip" simply is not there.

OPENAI'S SOLUTION

Randomize *everything*

Rubik's-cube paper (2019): randomize friction, cube size, finger geometry, gravity, tendon stiffness … roughly a hundred parameters. No single sim world could be exploited, because there was no single sim world.

Lesson — contact-rich tasks are the hardest to transfer, because contact is where every simulator lies the most. If your policy relies on a specific contact behaviour, assume it is exploiting the solver and randomize aggressively.

Diagnosing which gap you are facing

Does the policy work on hardware at all?

no

Ask — does it also work in a *second* sim (MuJoCo vs. PhysX)?
If no: your model relies on solver quirks.

Then — compare joint torques sim vs. real for the same trajectory. Big mismatch ⇒ actuator gap.

Then — playback sim rollout of the same policy. Does it look identical to real? If not, dynamics/contact gap.

partly

Ask — does the policy chatter, whine, or oscillate?
If yes: dynamics + action-smoothness.

Then — replay the recorded real observations back into the sim policy. If output differs, you have an observation gap.

Fix the identified gap → retrain → re-test.

Diagnose **one gap at a time**. Fixing three at once will make you think all of them mattered — and hide which one was actually decisive for your setup.

When the sim is lying

ISSUE 1

Reward keeps climbing but the video looks worse

A classic exploit signature — the reward function is being satisfied by an unphysical strategy. Watch a rollout, don't just read the plot.

ISSUE 2

Impossible torques or velocities

Log the max joint torque and max joint velocity across training. If they exceed hardware limits, the policy has learned a strategy that will simply not run on the robot.

ISSUE 3

Behaviour that only works with $d_t = 0.005$

If halving or doubling the physics time-step breaks the policy, the policy has learned physics that only your specific time-step supports. Real time is not tunable.

SZÉCHENYI EGYETEM
UNIVERSITY OF GYŐR

Bridging strategies

Part 4

JÁRMŰIPARI KUTATÓKÖZPONT
VEHICLE INDUSTRY RESEARCH CENTER

The toolkit

1

Shrink the gap

Make sim more like reality

- Better physical models
- System identification
- Learned actuator networks
- Higher-fidelity contact solvers
- Real2sim residuals

2

Survive the gap

Robustify the policy in sim

- Domain randomization
- ADR (auto DR)
- Observation noise & delay
- External perturbations
- Reward shaping for smoothness

3

Adapt across the gap

Let the policy identify reality online

- Domain adaptation
- Teacher–student distillation
- Rapid Motor Adaptation (RMA)
- Meta-RL
- On-robot fine-tuning

1. Better modelling

Before anything exotic, spend a day making the sim look more like your robot. These changes are boring, they take one afternoon each, and they cover a surprising fraction of the gap on their own.

✓ Weigh the real robot's links

Update mass and CoM in the URDF from measurement, not CAD.

✓ Match the sim d_t to control rate

50 Hz control ⇒ 20 ms action step; substeps at 500 Hz.

✓ Cap actuator torque and velocity

Set the same limits as the motor drivers enforce.

✓ Add sensor noise from the datasheet

✓ Measure control-loop latency

Round-trip a step command; use the number as the sim delay.

✓ Tune contact stiffness & damping

Drop a ball, match bounce height. Do the same in sim.

✓ Model the PD controller inside sim

Same K_p/K_d, same output filter.

✓ Match the observation vector exactly

Same fields, same order, same units, same estimation path.

> Do this **before** reaching for DR or RMA. If your sim's default friction is 1.0 and your foot is Teflon, no amount of adaptation will save you.

1. System identification

Formal name for "fit the sim to the robot". Run a scripted trajectory (chirps, steps, sinusoids) on the physical robot, record the state, then **fit simulator parameters** — friction, mass, motor gain, damping — so the sim reproduces that same trajectory.

The result: a "calibrated" simulator that is provably closer to *this* robot than the out-of-the-box model.

MINIMAL RECIPE

1. Record a chirp response on each joint.
2. Choose sim parameters θ (Kp, Kd, friction, motor gain).
3. Simulate the same input, get sim trajectory.
4. Minimize
$L(\theta) = \|q_{rea}(t) - q_{sim}(t; \theta)\|^2$
with CMA-ES / Bayesian opt / gradient.
5. Use θ* as the training simulator's defaults.

Trap — a perfectly identified sim is still an approximation. Sys-ID buys you a better *mean*, but does not eliminate variance. Always use it **with** randomization, never instead of it.

1. Actuator networks

When the actuator dynamics are too complex to model analytically (series-elastic joints, tendon-driven hands, custom drives), replace the analytical actuator model in the physics loop with a **small neural network** trained on real data.

The network learns the mapping (joint state, target) → actual torque, including hysteresis, thermal drift, and PD-loop delay. Slot it into the sim; the RL policy now trains against a much more realistic actuator.

Introduced for ANYmal in 2019; today used across quadrupeds, humanoids, and dexterous hands. Small (thousands of params), fast (single MLP evaluation per joint per step), and **learned once** per hardware revision.

2. Domain randomization

Instead of training on **one** simulation, train on a **distribution** of simulations — each episode samples a new set of physical parameters. The policy learns a strategy that works across the whole family. If the real world falls anywhere inside that family, it works there too.

SIM #1	SIM #2	SIM #3	REAL
μ = 0.3, mass +8%	**μ = 1.1, mass -5%**	**μ = 0.7, mass +2%**	**μ = 0.6, mass +1%**
1 ms latency	18 ms latency	6 ms latency	12 ms latency

If the policy handles SIM #1, #2, #3 and the thousands of others in between, then the real robot — whose parameters lie somewhere *within* the training distribution — is just one more sample.

Introduced by **Tobin et al., 2017** for vision; extended to dynamics by Peng et al. (2018) and popularised for robot manipulation by OpenAI's Rubik's-cube work (2019).

2. What to randomize

PHYSICS

The world's parameters

- Friction coefficient (per material)
- Restitution / damping
- Contact stiffness
- Gravity magnitude & direction
- Terrain height field
- Ground vibration

ROBOT

The robot's parameters

- Link mass, inertia, CoM offset
- Motor gains Kp, Kd
- Actuator torque limits
- Joint friction & damping
- Joint zero-offsets
- Control latency + jitter

SENSING & TASK

Everything the policy sees

- Observation Gaussian noise
- Observation delay + jitter
- IMU bias walk
- External push forces
- Payload mass on links
- Initial pose & velocity

A modern legged-robot recipe randomizes **30–100 parameters** per episode. OpenAI's hand paper randomized closer to 200.

2. Example – DR recipe

Parameter	Nominal	Sampled range	Distribution
Ground friction μ	0.7	[0.1, 1.5]	uniform, per episode
Ground restitution	0.0	[0.0, 0.3]	uniform, per episode
Base mass	m_0	± 15%	uniform, per episode
Base CoM offset	0	± 5 cm per axis	uniform, per episode
Motor K_p	nominal	× [0.8, 1.2]	log-uniform, per episode
Motor K_d	nominal	× [0.8, 1.2]	log-uniform, per episode
Motor torque scale	1.0	[0.9, 1.1]	uniform, per episode
Action delay	10 ms	[0, 25 ms]	uniform, per episode
Joint-pos obs noise σ	0	0.005 rad	Gaussian, per step
Joint-vel obs noise σ	0	0.5 rad/s	Gaussian, per step
Random push force	0	up to 150 N, 0.2 s	every 5–10 s
Extra link payload	0	[0, 5 kg]	random link, per episode

Ranges are illustrative — always slightly wider than what you expect the real value to be.

2. Automatic Domain Randomization (ADR)

Choosing DR ranges by hand is a black art — too narrow and the policy overfits; too wide and it never learns. ADR (OpenAI, 2019) removes the human from the loop:

1. Start with a narrow range around the nominal value.

2. Train — measure success on the current range.

3. When success is high, **widen the range** a bit.

4. When success drops, **narrow the range** again.

5. Repeat for each parameter independently.

WHY IT WORKS

The range widens until it hits the **edge of the policy's competence** — automatically curriculum-learning the hardest problem the policy can currently handle. No hyperparameter tuning of DR ranges.

OpenAI reported final ranges wider than any human would have written.

Trade-off — ADR trains far longer than fixed DR and is much more compute-hungry. In practice, most projects start with hand-tuned DR and only escalate to ADR when they hit a stubborn transfer failure.

2. When DR alone stops being enough

PROBLEM 1

The safety–performance trade-off

A policy trained on a very wide DR range is *safe* but *conservative*. It learns to compromise — the same walking gait for μ = 0.1 and μ = 1.5 is neither optimal nor stylish.

PROBLEM 2

Reactive, not identifying

Pure DR gives the policy no explicit way to **infer** the current parameter values. It reacts to whatever comes but never realises "oh, this floor is slippery" and adjusts.

PROBLEM 3

Unbounded gaps

Some real-world quantities cannot be enclosed in any reasonable range — sim-to-real texture gaps for vision, or fully-flexible morphology changes. DR runs out.

When DR alone stops being enough, add the third line of attack: **let the policy adapt to whatever specific reality it landed in**. That is the domain-adaptation family, the teacher–student trick, and its refined form RMA.

DR trains one policy for many worlds. Adaptation trains a policy that *notices* which world it's in.

3. Domain adaptation

Umbrella term for any technique that uses real-world signal — pixels, states, rewards — to adjust the policy or its inputs toward reality. Ranges from adjusting a single feature encoder to full on-robot fine-tuning.

FEATURE-LEVEL

Match feature statistics

Force sim-trained and real features into the same distribution (e.g. GAN-style discriminator, MMD loss). Cheap, common in vision Sim2Real.

OBSERVATION-LEVEL

Translate real → sim

Train a small image-to-image translator (CycleGAN etc.) that maps real camera frames to sim-looking frames, then feed those to the policy trained in sim.

POLICY-LEVEL

Online adaptation module

Add a small network that consumes recent state/action history and emits a latent describing "which world am I in" — RMA and its descendants.

WEIGHT-LEVEL

On-robot fine-tuning

Continue RL updates on the physical robot for minutes to hours. Sample-efficient methods (SAC, off-policy) preferred; the robot cannot burn a million episodes.

Modern legged-robot pipelines mostly use policy-level adaptation (RMA family). Manipulation Sim2Real still leans on feature-/observation-level translation for vision.

3. Teacher-student setup

Train **two** policies — one with cheat access to ground truth (the **teacher**), and one restricted to what the real robot can measure (the **student**) — then distill.

TEACHER (train in sim)

Privileged observations

Sees ground-truth friction, mass, exact terrain height around the feet, exact base velocity, external forces … things only the simulator knows.

Because it has all this information, it can be trained with normal RL (PPO) and reach very high performance across the whole DR distribution.

STUDENT (deploy on hardware)

Only what the robot senses

Sees only noisy joint encoders, noisy IMU, foot contacts — plus a **history** of past observations and actions.

Trained by **supervised distillation**: match the teacher's actions given only the non-privileged inputs. The student learns to infer the privileged information from history — implicit system identification.

This is the same idea underlying **RMA**, and directly used in most quadruped locomotion papers post-2021.

3. On-robot fine-tuning

Continue RL updates on the physical robot for minutes to hours after deployment. Simple in principle, treacherous in practice, because every gradient update costs a real trajectory.

Sample-efficient methods are non-negotiable — SAC / TD-MPC / offline warm-start; PPO's 100k-trajectory budget is a fantasy on hardware.

RULES OF THE ROAD

- Start from a sim-trained policy — never from scratch.
- Constrain the update to stay close to the pre-trained policy (KL penalty, LoRA).
- Kill-switch, torque limits, and a supervisor policy.
- Prefer offline / off-policy learning from a fixed dataset over live SGD.
- Log everything, so a bad update can be rolled back.

Most academic demos avoid it. Industrial deployments almost always end up doing *some* version of it — but they call it "calibration" and hide the SGD.

Tricks: Symmetry priors and reference motions

LEFT–RIGHT SYMMETRY

Halve the sample cost

Legged robots are almost mirror-symmetric. Augment every trajectory with its mirrored counterpart in the replay buffer, or bake symmetry directly into the network architecture (SE(2)-equivariant layers).

Effect — policy converges faster *and* generalises to the mirror image of every configuration it has seen.

REFERENCE MOTIONS

Anchor the exploration

Provide the policy with a periodic reference gait (from motion capture, a trot generator, or a CPG) and reward it for tracking + reward for original task.

Prevents the policy from wandering into unphysical exploration and reliably produces natural-looking gaits that transfer well.

Both of these are essentially **injected inductive biases** — they narrow the space of behaviours the policy explores, which in turn narrows the space of sim-artifacts it can exploit.

Tricks: Add noise, delay, jitter

NOISE

Match the datasheet σ

Add Gaussian noise to every observation channel, magnitude drawn from the sensor spec. Bias per-episode, white noise per-step.

DELAY

Feed a stale observation

Buffer past observations; return the one from k steps ago, with k sampled per episode across the measured hardware range.

JITTER

Random extra frames

Real control loops occasionally miss a deadline. Skip a policy step every few seconds; hold the previous action.

A policy trained on clean, on-time observations has never learned to filter or to tolerate delay. It behaves like a very confident driver with a lag in the steering wheel — which is exactly what a real robot is.

A commonly-forgotten detail — noise the **observation**, not the ground-truth state used for reward. Reward stays clean; only the input to the policy is corrupted.

Which strategy should be applied?

Technique	What it addresses	Cost
Better modeling	Any static-parameter gap	Low, one-time
System identification	Nominal parameter values	Low, per hardware unit
Actuator networks	Complex motor dynamics	Medium, per hardware revision
Domain randomization	Parameter-value uncertainty	Medium (training compute)
ADR (auto)	DR range hyperparameters	High compute
Observation noise / delay	Sensing pipeline gap	Trivial
Reward shaping	Exploits + hardware safety	Low
Teacher–student	Uses privileged info during training	Medium (2 training phases)
RMA	Online identification	Medium
On-robot fine-tune	Residual unmodelled gap	Very high (real time on robot)

SZÉCHENYI EGYETEM
UNIVERSITY OF GYŐR

Thank you for your attention!

JÁRMŰIPARI KUTATÓKÖZPONT
VEHICLE INDUSTRY RESEARCH CENTER